

What Is Different — GCS: UAVXGUI → UAVXGS

Platform

Aspect	Old UAVXGUI	New UAVXGS
Language	C# (.NET Framework)	Python 3
UI toolkit	Windows Forms	Qt5 (PyQt5)
Runs on	Windows only	Linux, macOS, Windows
Installation	MSI installer	<code>git clone && python3 main.py</code> or single .exe / .app

Impact: The old GCS was tied to Windows. The new one runs identically on all three platforms with the same source code. No Mono, no Wine, no VM needed.

Connection

Old	New
Manual COM port entry Fixed 115200 baud No auto-reconnect	Dropdown with auto-detect + editable field Selectable: 9600–128000 Status-monitored; reconnection attempt on link loss

Impact: Less fiddling. Plug in the FC, pick the port from the list, click Connect.

Main Instrument Panel

Old	New
2D artificial horizon (GDI) Separate compass widget Three separate alt/ROC/speed tapes Fixed-size window Dark grey background	3D attitude sphere with compass rose Compass integrated into attitude sphere rim Compact altitude box with large numeric ROC Resizable; attitude sphere scales with window Black background with green-on-black instruments (high-contrast)
Debug log in separate window	Debug messages in status bar with verbosity filter

Impact: Cleaner, more compact layout. All essential flight data visible without scrolling. The attitude sphere is larger and more readable.

Parameter Window — The Legacy Checkbox Is Gone

This is the single biggest usability change.

Old System

The old UAVXGUI stored parameters in a mix of `float32` and `uint8`. Some parameters (gains, rates) were 0-255 integers on the FC but needed to be displayed as meaningful values (e.g. 0-100%). This required a **Legacy checkbox** in the GCS that toggled the display between the raw `uint8` integer and a converted float. There was also a “Legacy Tag” column showing the conversion formula.

Problems with the old system: - You had to know whether a given param was legacy or not - The checkbox affected the display but not the underlying value — confusing - Saving and reloading a param file could silently lose precision if you had the wrong mode selected - Adding a new parameter meant updating conversion tables in both FC and GCS

New System

All 128 parameters are `float32` — no exceptions. The FC stores and transmits every param as 4 IEEE-754 bytes. The GCS applies `PARAM_DISPLAY_MULT[idx]` to convert between FC-native units and the display value, uniformly for all parameters.

What disappeared: - **Legacy checkbox** — completely removed - **Legacy Tag column** — removed - **uint8 display conversion** — removed - **PARAM_SCALES** — removed from both FC and GCS - **LEGACY_TAGS** — removed from both FC and GCS

What stayed the same: - Parameters still have per-index display multipliers and limits - The FC still stores the same effective range (0-255 for former `uint8` params, now as `(float32)(uint8)value`) - Airframe `.af` files still load and save the same way

What This Means

Old Behaviour	New Behaviour
See “150” in a <code>uint8</code> field, wonder if it’s 150 raw or 150%	Always see the display value directly
Click Legacy checkbox to see “75.0%” instead of “150”	No checkbox needed — you always see “75.0%”
Save params, reload, get a different number because the mode was wrong	Save/load is always consistent
Need to read the source to know if a param is <code>uint8</code> or float	All params behave identically

Old	New
Two-page layout (Basic + Advanced)	Single table with category tabs
Mixed <code>uint8</code> + float display	Unified <code>float32</code> — every param reads/writes as a decimal
Legacy checkbox to switch <code>uint8</code> ↔float	Removed — all params are always float
“Legacy Tag” column present	Removed — <code>LEGACY_TAGS</code> and <code>PARAM_SCALES</code> deleted
Manual refresh per page	One Read button loads all 128 params
Parameter save in proprietary format	<code>.af</code> files — readable tagged format

Navigation Window

Old	New
Bing Maps (requires API key)	OpenStreetMap (free, no key)
Map tiles loaded synchronously	Async tile loading via QWebEngineView
No GPS track log	Built-in KML export
No telemetry CSV log	Built-in CSV flight log
Mission only editable via right-click	Add/delete/clear buttons + right-click menu

Impact: No API key to obtain. Maps work offline (tile caching). Flight logs are automatic.

Calibration

Old	New
Separate IMU and Mag buttons	Combined calibration window
LED indicators on main window	Status LEDs per sensor + live readback
No sensor data preview	Live sensor values shown during calibration

Impact: Clearer feedback during calibration. You can see the sensor reading change as you move the aircraft.

New Features (Not in Old GCS)

Feature	Description
DFU Flasher	Built-in firmware upload via UART DFU — no external programmer needed
Raw Packet Monitor	Scrollable hex dump of every protocol packet — invaluable for debugging
Tuning Packet Display	Rate tracking data (desired vs actual rate per axis) available in real-time
Voice Feedback	Text-to-speech for events (armed, RTH, low battery) via espeak/SAPI/NSSpeechSynthesizer
KML Track Export	Automatic KML file generation during flight — view in Google Earth
CSV Flight Log	Structured CSV with all telemetry fields at ~10 Hz
AttitudeControl removal	Tag 68 per-axis sends gone — replaced by single tuning packet (tag 57) every cycle

Impact: The new GCS does everything the old one did plus DFU flashing, logging, speech, and protocol debugging. No external tools required.

Removed (No Longer Needed)

Feature	Reason
Legacy param checkbox	All params are float — no conversion needed
Bing Maps API key setup	Switched to OpenStreetMap
.NET Framework / Mono dependency	Python runtime is self-contained
Windows-only TTS (SAPI)	Cross-platform TTS via pyttsx3
Separate UAVXSet tool	Parameter management merged into GCS

Summary

If you used the old UAVXGUI, you will find: - The same instrument data, displayed differently (arguably better) - Parameter editing without the float/uint8 confusion - A map that works without an API key - Built-in logging and DFU flashing (was external tools) - Your GCS now runs on your Mac or Linux laptop too